COMP2511

Tutorial 8

Last Tutorial

- Abstract Factory Pattern
- Decorator Pattern
- Singleton Pattern

This Tutorial

- Introduction to Software Architecture
- Software Architecture Diagrams
  - C4 Diagrams
  - Sequence Diagrams

Software Design

- Software ***design*** is focused on how things work at the code-level, and how individual parts of a system are **implemented**.
  - This includes detailed decisions about the classes we model, the methods we write and the design patterns we decide to use.

Software Architecture

- Software ***architecture*** operates at a macroscopic level and involves decisions relating to the fundamental high-level structure of a system. This involves:
  - how the system is divided into modules (components), services and layers
  - how data moves between different components
  - which technologies, frameworks and infrastructure are used
  - considerations about traits such as security, scalability, performance of applications - these are referred to as architectural ***characteristics***.

Design vs. Architecture

- When building a house, design decisions include choosing wallpaper, selecting different types of furniture, etc.
- Architectural decisions guide the overall structure of the house, such as the foundation, layout of the rooms, roofing etc.
- Architectural decisions are much harder to change than design decisions.

Typical Application Requirements

- A typical application needs:
  - a way for the user to interact with the application (the ‘front-end’),
  - components that process user requests and perform the required logic (the ‘back-end’) which is connected to a data-store.
- We will look at two types of diagrams used to communicate how a system interacts with users and processes logic.

C4 Model

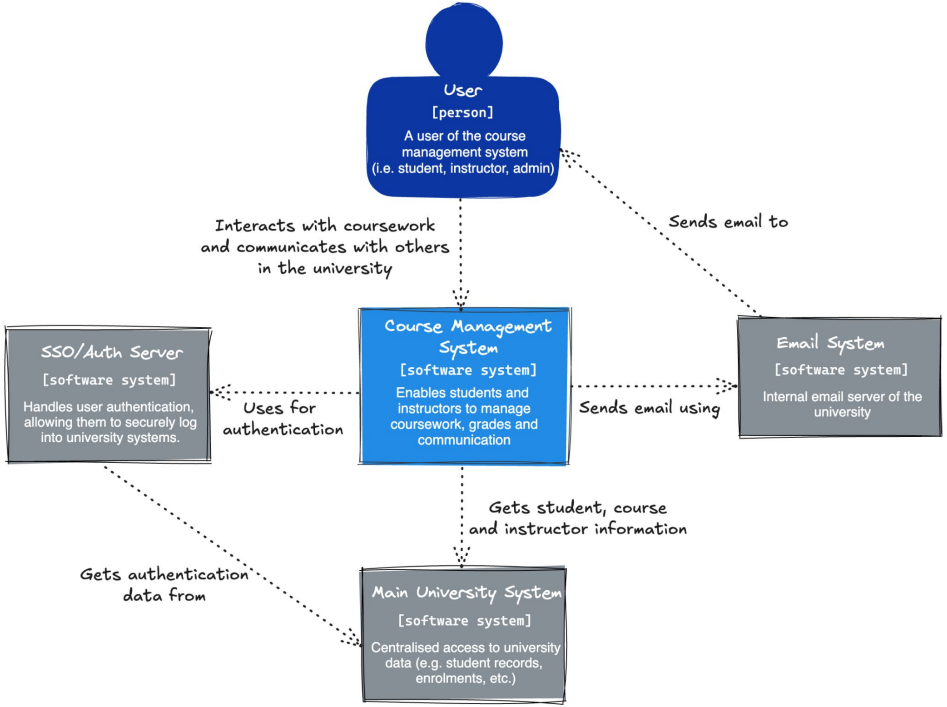
- The C4 Model is a framework for visualising the components of a software system at **four** different levels of abstraction.
- Each level is intended for different people; a high-level overview of the system is good for stakeholders, while developers need low-level details.
- The four levels of the C4 Model in decreasing order of complexity:
  - **Context Diagram**
  - **Container Diagram**
  - Component Diagram: Breaks a specific container into its major components; not relevant for the course.
  - Code (Class) Diagram: Zooms into a specific component to show details of its source code (e.g. what classes are used); UML class diagrams!

Context Diagrams

- A Context Diagram communicates the overall function of the application, how people would use it and which existing systems the application interacts with.
- It is meant to be a simple and accessible overview of the system directed towards people with little to no technical background.

Context Diagram: Example

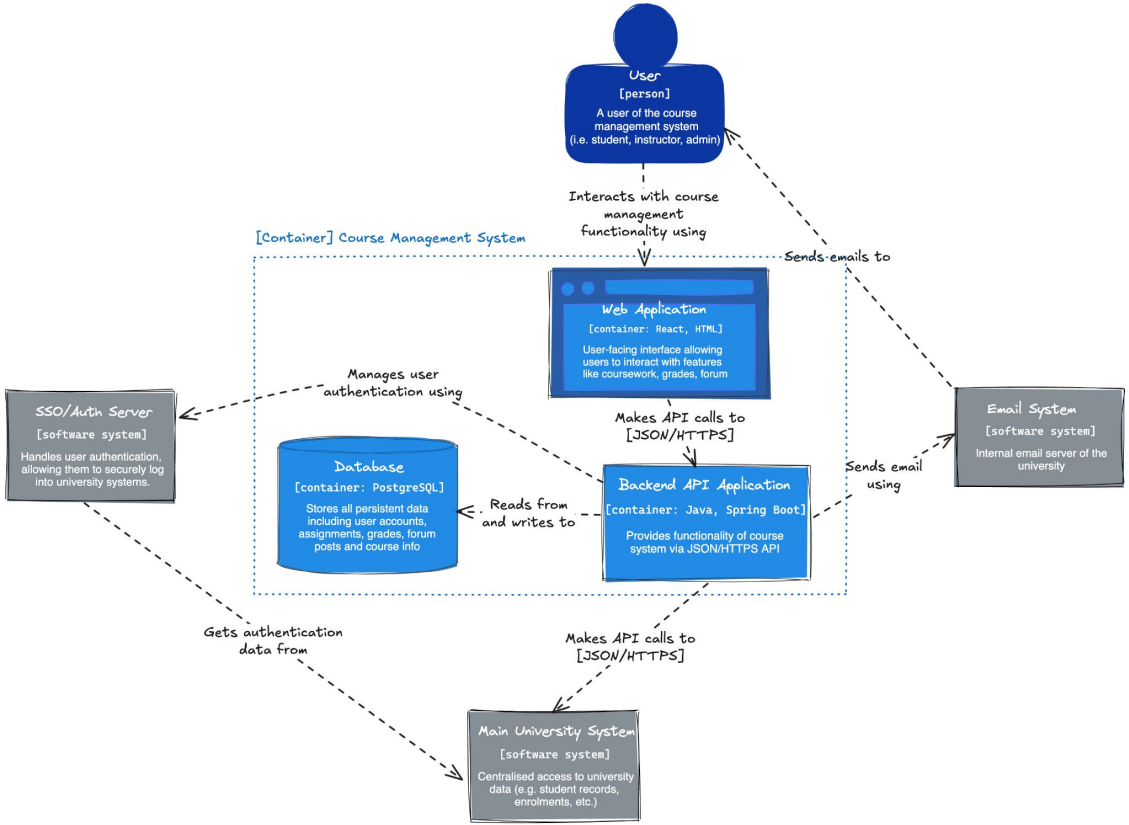
- **Example:** You're designing a course management system (like Moodle) used by students and instructors.
  - Students can view and submit assignments, receive grades and participate in forums
  - Instructors can create and grade assignments, post announcements and manage students
  - The system has a web frontend, backend, database and integrates with other university systems like authentication and email.



Container Diagrams

- A Container Diagram breaks the system into its major components (referred to as containers) and how they interact with each other.
- The Container Diagram is basically a Context Diagram, but you are **zooming into the software system** being modelled.
  - This means breaking the system down into its major deployable units (e.g. web apps, mobile apps, databases, *microservices* etc.) and showing how they communicate with each other.
  - Everything else is the exact same as the Context Diagram.

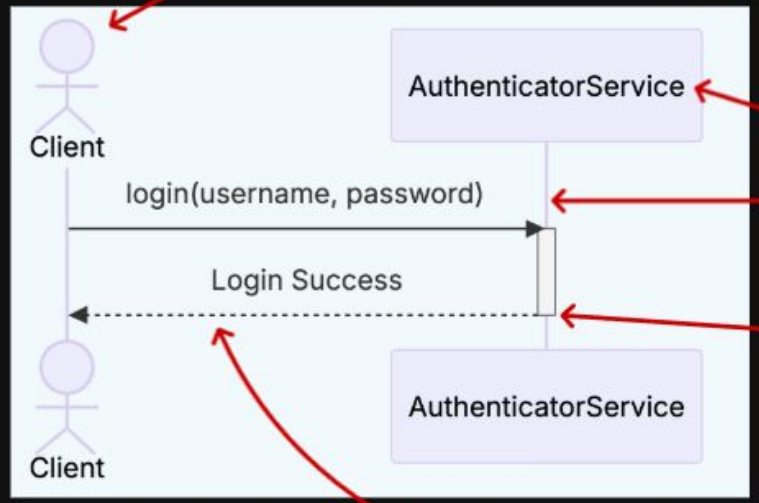
Container Diagram: Example



Sequence Diagrams

- A Sequence Diagram illustrates the flow of communication between components to achieve a specific functionality or use case.
- It is suitable for visualising the flow of the application over time in detail.
  - This is contrast to a **class diagram** (e.g. UML diagrams), which show the **static** structure of a system. These describe *what* the system is made of, while sequence diagrams describe *how* the system behaves over time.
- Sequence Diagrams often do not show the entire system; each sequence diagram models just one specific functionality of the system.
- Course Notes on Sequence Diagrams - please make use of this resource as this explains all of the required notation and syntax!

Sequence Diagrams: Key Elements (credit to Rebecca!)



Actor: External user or system

Objects: Entities of the system

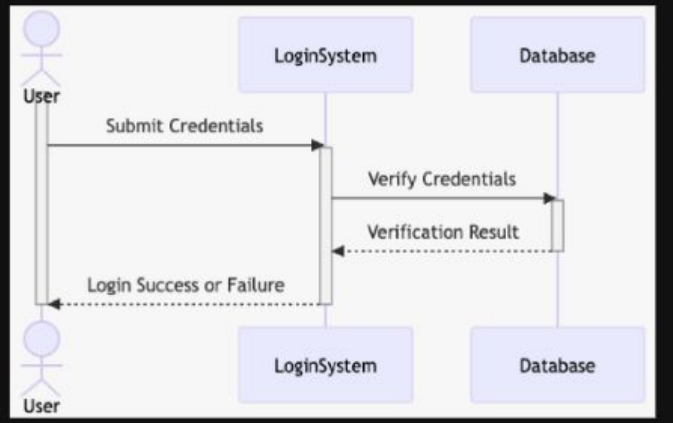
Lifelines: shows object existence during interactions

Activation boxes: indicates active processing of messages

Messages: Communication between objects

Sequence Diagrams: Axes (Slide from Rebecca)

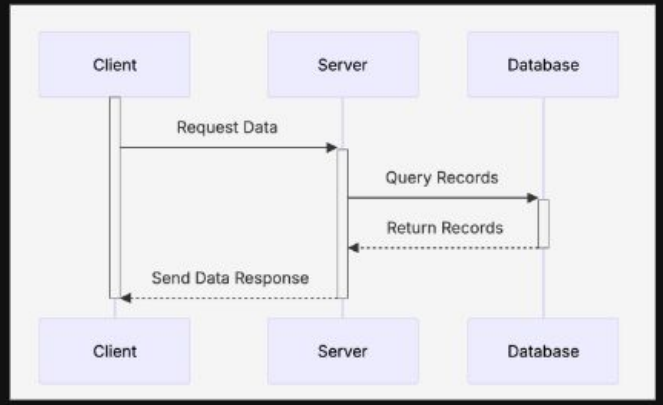
- Horizontal axis represents objects
  - Objects placed left to right
  - The order represents the message sequence
- Vertical axis represents time
  - Time flows downward
  - Sequence diagrams prioritise order, not duration
  - Therefore, vertical spacing does not represent any sort of time intervals



Sequence Diagrams: Messages (Slide from Rebecca)

Synchronous

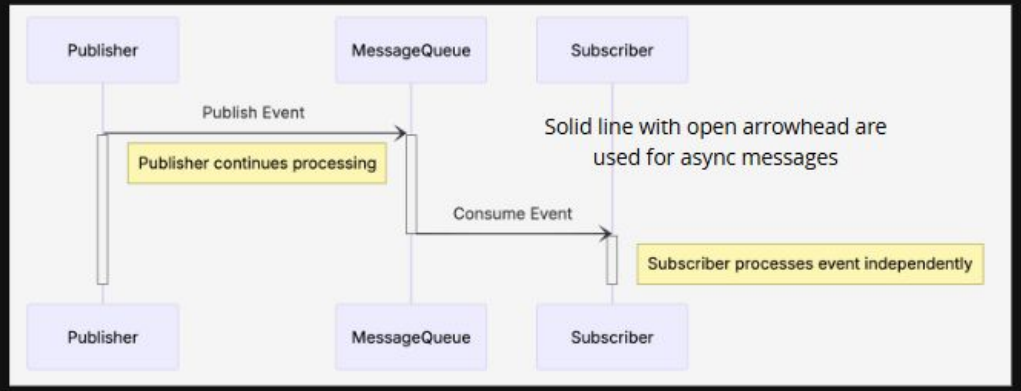
Sender waits for the receiver to complete the operation and return a response before continuing its own execution



Activation box exists so the message is processed immediately

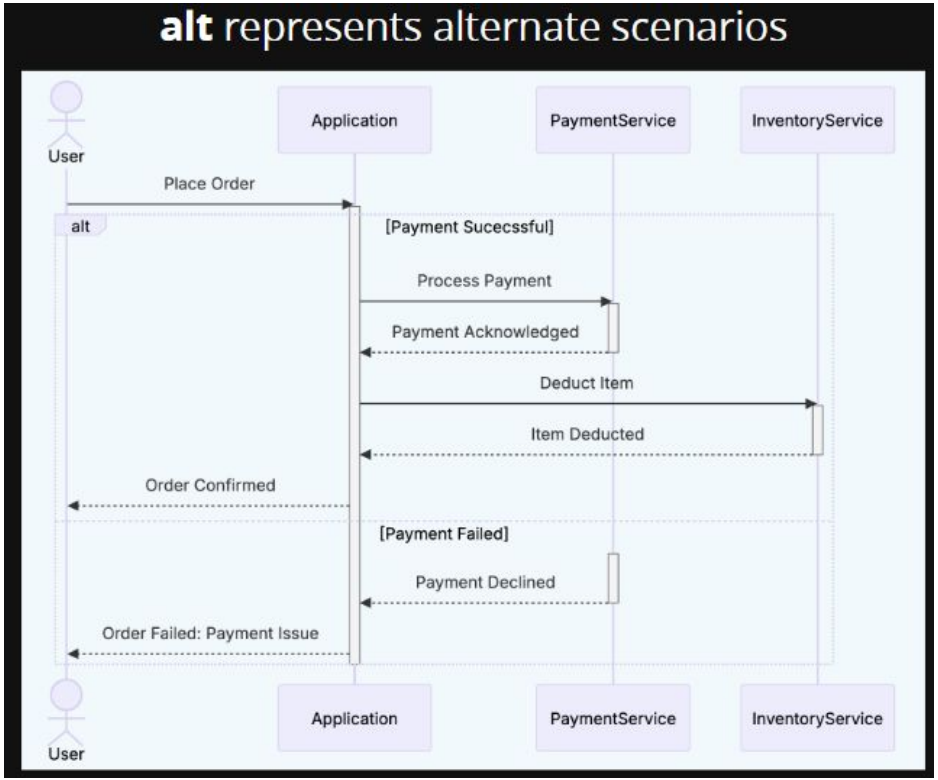
Asynchronous

Sender does not wait for the receiver to complete the operation; it sends the message and continues its own execution

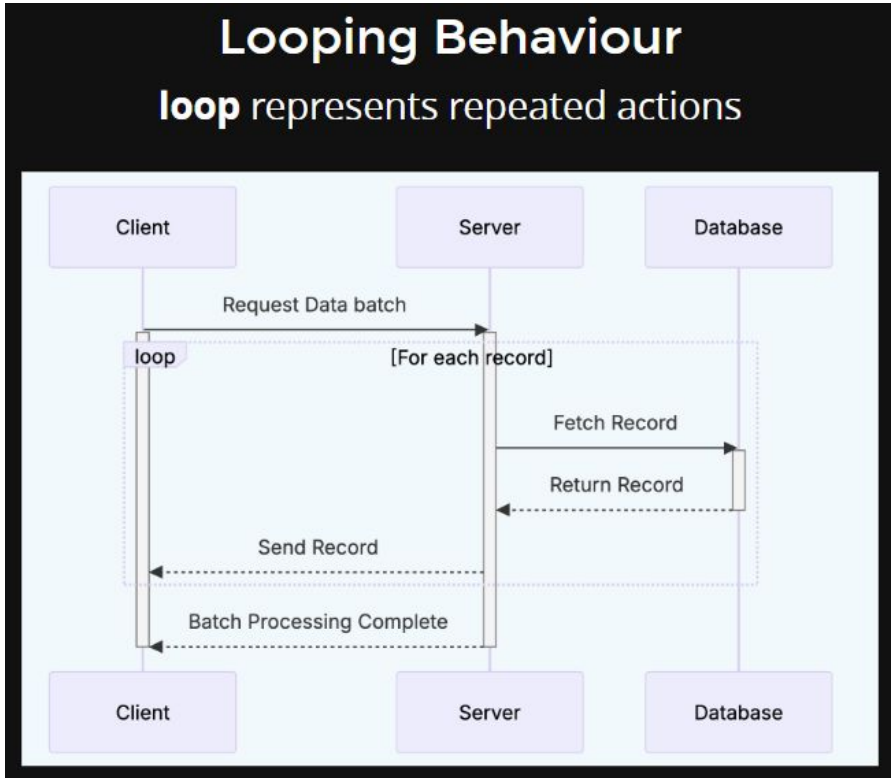


No activation box so message processing and response is delayed

Sequence Diagrams: Conditionals (Slide from Rebecca)



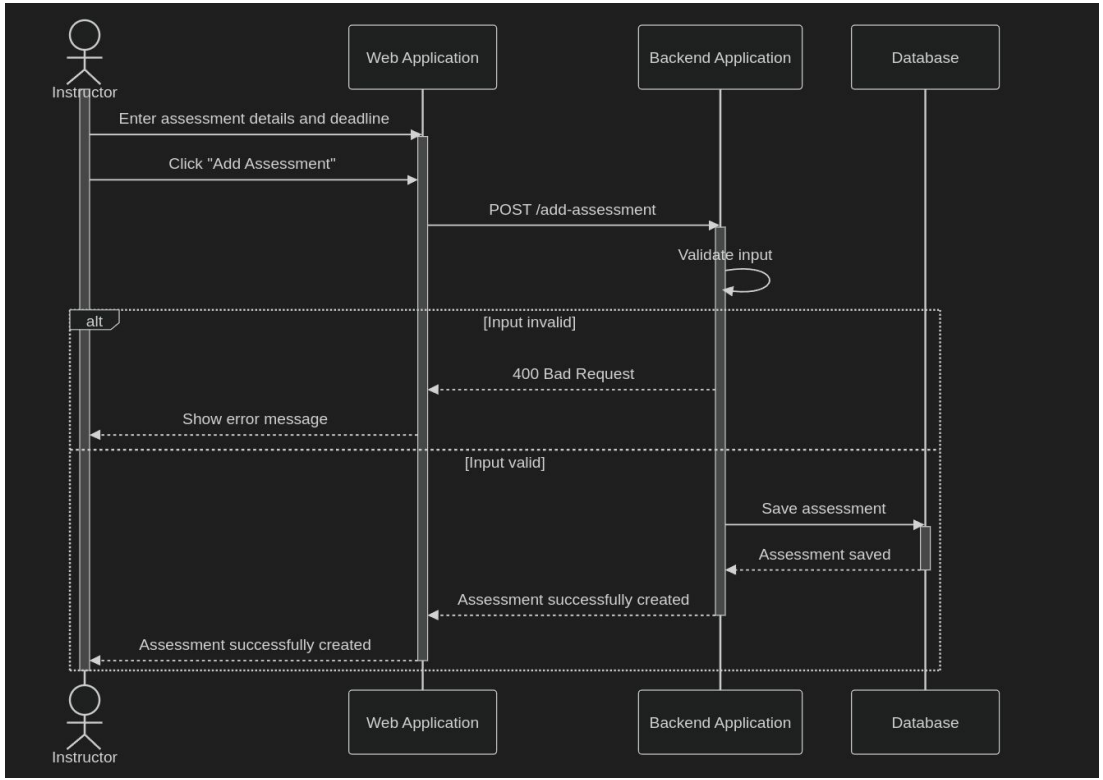
Sequence Diagrams: Loops (Slide from Rebecca!)



Live Example: Sequence Diagrams

- **Example:** You are part of the development team designing the Course Management System. Create a Sequence Diagram illustrating the flow when an instructor making an assessment with a deadline.
  - Instructor creates a new assessment by clicking on “Add Assessment” button on the Web Application
  - Web Browser sends data to the Backend API Application
  - Backend validates input
    - If input invalid, Backend sends to frontend 400 Bad request
    - Else, continues with saving the assessment
  - Backend attempts to save the assessment data to Database
  - Database confirms save
  - Backend sends confirmation of successful assessment creation to Frontend
  - Frontend provides result notification to the Instructor

Sequence Diagram: Example



Lab 8 Submission

- You will need to make C4 Diagrams and Sequence Diagrams for Lab 8. Please remember to submit these on **Moodle**.
  - Submissions have a 20 word minimum, which should be met as you will also have to answer a short answer question in your submission.